Neural World Model & Predictive Robot Intelligence

Technical report | Learned dynamics, obstacle forecasting, multi-future MPC

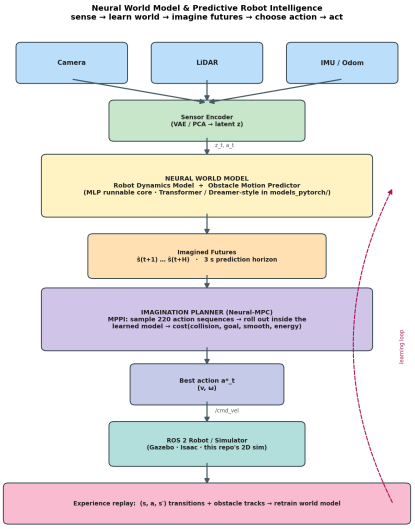
Project: CPU-based learned world models for mobile-robot navigation among moving obstacles. **Scope:** synthetic 2D simulation; not Gazebo/Isaac Sim or physical-robot validation.

Abstract

A robot-dynamics MLP and a rotation-invariant obstacle-motion predictor are trained on simulated experience and used inside sampling-based model-predictive control. A five-member bootstrapped ensemble supports multiple possible obstacle futures and a risk-sensitive CVaR planner. In the 25-episode-per-planner stress evaluation reported here, ensemble-mean Neural-MPC achieved 68.0% success and 24.0% observed collision frequency, while Risk-MPC achieved 76.0% success and 12.0% observed collision frequency. These are measured outcomes from a small synthetic benchmark, not guarantees of robot safety.

System and methodology

A unicycle robot navigates a 20 m x 20 m simulated arena with moving circular obstacles, curved/noisy motions and wall interactions. The learned robot dynamics and history-conditioned obstacle model provide state rollouts. Sampling-MPC evaluates candidate action sequences using collision, proximity, goal, energy and smoothness penalties. Risk-MPC compares candidate actions across the ensemble and emphasizes higher-cost futures.



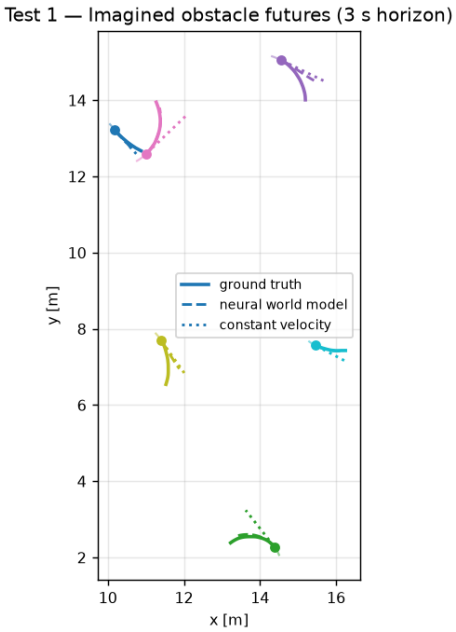
Architecture of learning, imagined futures and planning.

Evidence source: results/benchmark_results.json and repository-generated figures. This PDF is a regenerated presentation of stored results, not a new training run.

Test 1 | Dynamic obstacle forecasting

The baseline compares constant-velocity extrapolation against the learned obstacle-motion model over a 3-second prediction horizon. Lower ADE/FDE is better.

Predictor	ADE in [m]	FDE in [m]	ADE unseen [m]	FDE unseen [m]
Constant velocity	0.190	0.522	0.448	1.231
Neural obstacle model	0.121	0.335	0.187	0.522



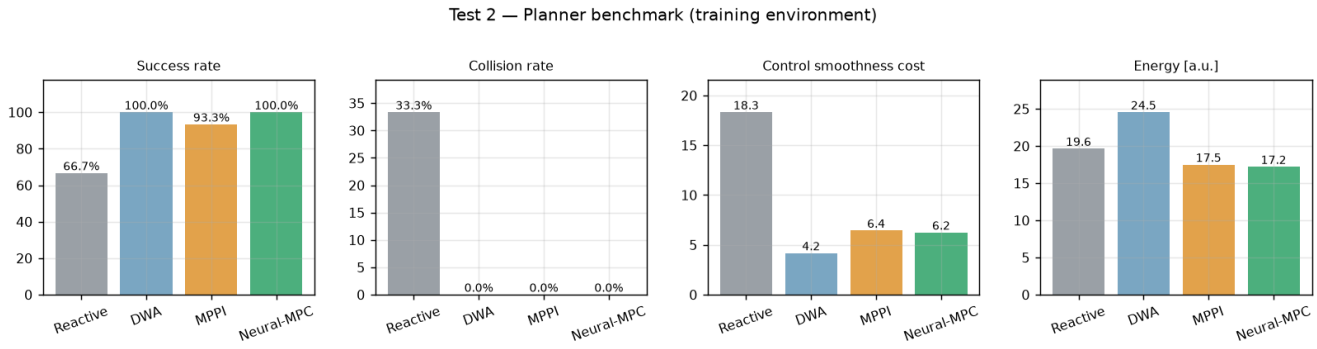
Example of forecasted obstacle paths versus ground truth.

Interpretation: The learned predictor reduces prediction error in both reported evaluation environments. Error around wall bounces and distribution shift remains a limitation. Obstacle positions are supplied by simulation rather than estimated from real sensors.

Tests 2 and 3 | Planning and generalization

The base environment and a harder unseen environment each evaluate the reactive, DWA-style constant-control, analytic-MPPI and learned-model Neural-MPC planners. Success and collision are empirical episode fractions.

Planner	Success: base	Collision: base	Success: unseen	Collision: unseen
Reactive	66.7%	33.3%	70.0%	30.0%
DWA-style	100.0%	0.0%	96.7%	3.3%
Analytic MPPI	93.3%	0.0%	93.3%	0.0%
Neural-MPC	100.0%	0.0%	93.3%	0.0%



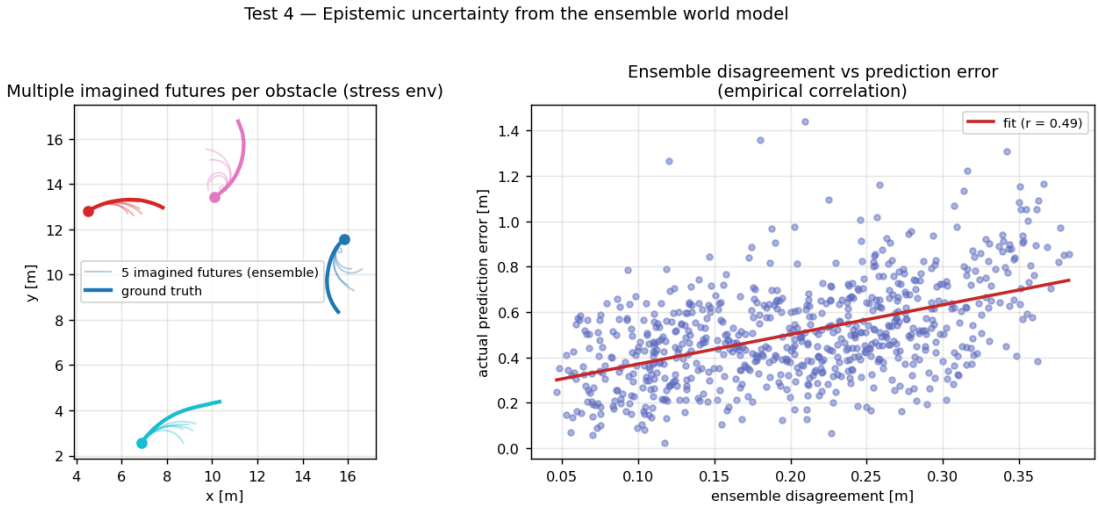
Base-environment planning benchmark.

For the analytic MPPI versus Neural-MPC ablation, the optimizer and cost terms are shared, while the dynamics and obstacle prediction model change. Reactive and DWA-style planners are different algorithms, so claiming identical cost across all four is not justified.

Tests 4 and 5 | Model disagreement and risk-aware planning

Test 4 | Empirical disagreement-error association

Across the reported 720 obstacle tracks, ensemble disagreement has a correlation of r = 0.492 with realized prediction error. This is a positive association, not a formal uncertainty-calibration test.



Five predicted futures and empirical disagreement versus error.

Test 5 | Stress evaluation with 12 moving obstacles

Planner	Success	Collision	Mean energy
Ensemble-mean Neural-MPC	68.0%	24.0%	17.20
Risk-MPC (ensemble CVaR)	76.0%	12.0%	18.84

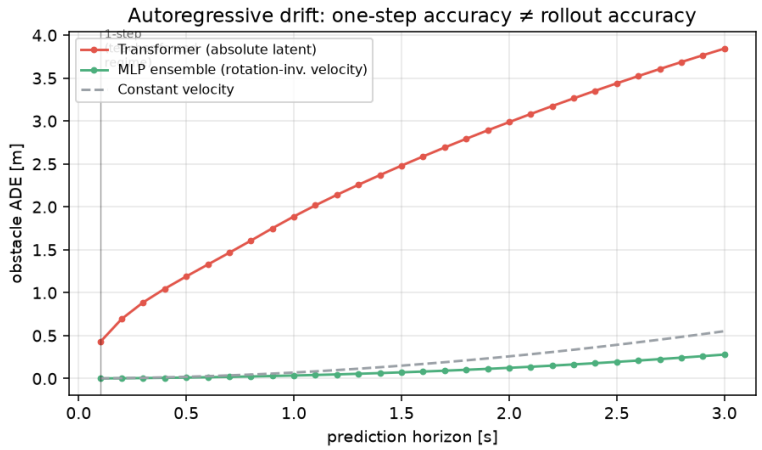
These two planner results came from 25 simulated episodes each, as verified in the local Stage 2 console log. The unified JSON currently does not store that episode count. No single-model baseline was evaluated in this updated run.

Test 7 | Transformer rollout stability

A sequence Transformer and a rotation-invariant MLP ensemble were compared on long-horizon prediction in the reported simulation.

Predictor	3-second ADE [m]	3-second FDE [m]
MLP ensemble	0.100	0.280
Transformer	2.390	3.886
Constant velocity	0.202	0.552

The preliminary Transformer experiment shows substantial autoregressive drift. The original evaluation also has methodological limitations (including sequence-window splitting and different input/rollout conventions), so the numbers should not be presented as a definitive architecture comparison.



Saved model-prediction drift visualization.

Test 6 | Occupancy prediction

The repository contains occupancy figures and code, but this unified benchmark JSON does not contain Test 6 metrics. This report deliberately does not reproduce unverified numerical claims for Test 6.

Limitations and reproducibility

Simulation scope. The environment is a compact 2D simulator with tracked obstacle coordinates and a simplified robot model. The work has not established end-to-end real-robot perception or safe physical deployment.

Metrics. All success and collision rates are observed within the specified simulation episodes; zero observed collisions does not imply zero collision probability. Stress results use small sample sizes, and comparisons with different evaluation seeds or model checkpoints require care.

Model limitations. The obstacle model relies on ordered, known obstacle tracks. It has limited treatment of interactions, identity switches, uncertain state estimates, and physical safety layers.

Reproduction. In a clean checkout, install requirements with the tested CPU constraints, run the base model/pipeline, then the ensemble upgrade stages. The base pipeline merges its own results into the unified JSON; historical sections should not be taken as results of a single fresh training run unless they have been reproduced together.

Hardware transfer. ROS 2 integration is an experimental scaffold. Gazebo, Isaac Sim, sensor integration and hardware safety validation remain future tasks. The 2D simulator is not a substitute for those tests.

Data and provenance

Benchmark metrics: results/benchmark_results.json. Figures: assets/. Source scripts: scripts/run_pipeline.py, scripts/run_upgrade.py, scripts/run_stage2_benchmark.py. All metrics in the tables above are read from the supplied JSON; the Test 5 episode count is based on the separately recorded local Stage 2 run.

Generated: 2026-10-17 | Document generated by scripts/make_technical_report.py